

DOCUMENTAÇÃO TÉCNICA · ALTERAÇÃO DOCUMENTADA

Cache no core e reuso dos casos de uso

Caches em memória com TTL de cinco minutos para lista e detalhe

Comparação: commit f7d49ae (HEAD) x working tree

Branch: v2-rastreamento

Data: 29 de julho de 2026

Arquivos principais: core/domain/tracking/tracking.ts; tracking-detail.ts

Resultado: chamadas repetidas podem ser respondidas pelo processo Node sem consultar novamente o portal SSW.

Cache da listagem

```
Map<cpfNormalizado, { data: TrackingListItem[]; expiresAt: number }>  
TTL = 5 minutos
```

1. Validar e normalizar o CPF.
2. Consultar `listCache`.
3. Retornar a lista se ainda estiver válida.
4. Caso contrário, executar scraping e parsing.
5. Armazenar o resultado com nova expiração.

Cache do detalhe

```
Map<cpfInformado:trackingId, { data: TrackingDetailResponse; expiresAt: number }>  
TTL = 5 minutos
```

O detalhe verifica o cache antes de carregar a lista. Em caso de miss, usa `fetchTrackingListByCpf`, encontra o item, consulta o detalhe e armazena o envelope completo.

Deduplicação

`fetchTrackingListByCpf` deixa de ser privado e passa a ser reutilizado por `tracking-detail.ts`. Validação, scraping e parsing da lista ficam em uma única implementação.

Efeito nas chamadas externas

Cenário	Sem cache	Com cache no mesmo processo
Nova listagem	2 chamadas	2 chamadas
Listagem repetida em 5 min	2 chamadas	0 chamadas
Primeiro detalhe após lista quente	3 chamadas	1 chamada
Detalhe repetido em 5 min	3 chamadas	0 chamadas

Limitações

- Cache local ao processo: não é compartilhado entre instâncias serverless.
- É perdido em restart ou cold start.
- Não há limite global de chaves nem limpeza periódica.
- O detalhe usa CPF apenas com `trim` na chave, podendo fragmentar formatos equivalentes.
- Respostas do cache preservam o `scrapedAt` antigo durante o TTL.
- CPF e dados de rastreamento permanecem na memória do processo.